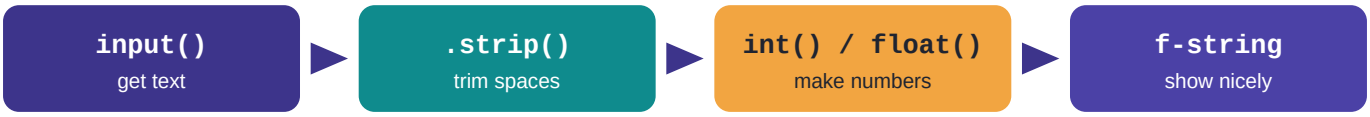
DAY 2

Input Formatting and String Formatting

4-page quick summary | for students new to Python

The big idea: input() always gives you text. Clean it, convert it to a number if you need maths, then print it neatly.



1 Reading input and cleaning it with strip()

`input()` pauses the program, shows a prompt and returns what the user types **as text**. `.strip()` removes spaces from the **start and end** only.

```
name = input("Enter a name: ").strip()
print("Welcome,", name)
```

```
OUTPUT
Enter a name: Asha
Welcome, Asha
```

Method	What it trims	Result on " Python class "
`.strip()`	both ends	`"Python class"`
`.lstrip()`	left side only	`"Python class "`
`.rstrip()`	right side only	`" Python class"`

REMEMBER Python is **case-sensitive**: `input()` works, `Input()` does not. Parentheses matter: `.strip()` is a method call.

2 Formatting data and data type mismatch

Typing digits does not make a number. The value arrives as **text (str)**.

```
number = input("Enter your number: ")
print(f"Hello {number}", type(number))
```

```
OUTPUT
Enter your number: 25
Hello 25 <class 'str'>
```

PREDICT If `number = "25"`, then `number + number` gives the **text** `"2525"` (joined), not the number 50.

3 Convert input: int() and float()

- `int("25")` gives the integer **25** | `float("25.5")` gives the decimal **25.5**
- `int("25.5")` raises **ValueError** (not an integer). Use `float()`, or `int(float("25.5"))` to truncate.
- Whole numbers `int()`, decimals `float()`. Wrap in **try / except** to handle bad input.

```
age_text = input("Enter your age: ").strip()
try:
    age = int(age_text)
except ValueError:
    print("Please enter a whole number, such as 18.")
else:
    print(f"Next year you will be {age + 1}.")
```

```
OUTPUT
Enter your age: abc
Please enter a whole number,
such as 18.

Enter your age: 18
Next year you will be 19.
```

TRY IT To accept an age like 18.5, swap `int(age_text)` for `float(age_text)`.

4 Output formatting with print()

`print()` separates items with a space and ends with a new line. Change that with `sep=` and `end=`.

```
print("Python", "Basics", "Day 2")
print("2026", "10", "13", sep="-")
print("Loading", end="...")
print("Done!")
```

```
OUTPUT
Python Basics Day 2
2026-10-13
Loading...Done!
```

Option	What it controls	Default
`sep=`	text placed **between** the items	one space
`end=`	text printed **at the end**	new line (\n)

PREDICT `print("a", "b", "c", sep="")` prints `abc` (nothing between the items).

5 f-strings: readable formatted output

Put an **f** before the quote and variables or expressions inside **{ }**. This is usually the clearest way to build messages.

```
name = "Asha"
age = 19
marks = 87.456
print(f"{name} is {age} years old.")
print(f"Marks: {marks:.2f}")
print(f"{name} scored {marks:.1f} marks.")
```

```
OUTPUT
Asha is 19 years old.
Marks: 87.46
Asha scored 87.5 marks.
```

Format	Meaning	Example result
`{marks:.2f}`	fixed-point, **2** decimal places (rounded)	marks = 87.456 gives 87.46
`{marks:.1f}`	fixed-point, **1** decimal place	marks = 87.456 gives 87.5
`{pct:.1%}`	multiply by 100, add %, 1 decimal place	pct = 0.875 gives 87.5%
`{a + b}`	any expression works inside the braces	a = 2, b = 3 gives 5

KEY POINT Formatting changes only **how a value is displayed** (as text). The variable still holds the original value, e.g. 87.456.

Three ways to build the same message (name = "Asha", age = 19):

Style	Code	Note
+ and str()	`print(name + " is " + str(age) + " years old.")`	numbers must be converted with str()
commas	`print(name, "is", age, "years old.")`	spaces are added for you
f-string	`print(f"{name} is {age} years old.")`	clearest; no conversion needed

The older .format() style still works, but f-strings are easier to read:

```
name = "Ravi"
marks = 91.5
print("{} scored {} marks.".format(name, marks))
print("{:.2f}".format(marks))
```

```
OUTPUT
Ravi scored 91.5 marks.
91.50
```

6 Basic string operations

A string (**str**) is text in quotes. Strings are **immutable**: methods return a **new** string instead of changing the original.

Operation	Example	Result
+ join	`first_name + " " + last_name`	`Asha Patil`
* repeat	`"Hi! " * 3`	`Hi! Hi! Hi!`
len() count characters	`len(full_name)`	`10`
[0] first character	`full_name[0]`	`A`
[-1] last character	`full_name[-1]`	`l`

Indexing starts at **0**, and **-1** is the last character.

Handy method	What it does	Example
`.upper() / .lower()`	all capitals / all small letters	"Hi".upper() gives "HI"
`.title()`	Capital Letter On Each Word	"asha patil".title() gives "Asha Patil"
`.split(",")`	cuts text into a **list** at each comma	"a,b,c".split(",") gives ['a', 'b', 'c']

TRY IT For `city = "Pune"`: first character is `city[0]` (P), last character is `city[-1]` (e).

7 Mini project: student marks report

Combines input, type conversion, string cleanup and formatted output.

```
name = input("Student name: ").strip()
subject = input("Subject: ").strip()
marks = float(input("Marks obtained: "))
total_marks = float(input("Total marks: "))

percentage = marks / total_marks
print("\n--- Student Report ---")
print(f"Name       : {name}")
print(f"Subject    : {subject}")
print(f"Marks      : {marks:.1f} / {total_marks:.1f}")
print(f"Percentage : {percentage:.1%}")
```

```
OUTPUT
Student name:   Asha
Subject: Maths
Marks obtained: 87.5
Total marks: 100

--- Student Report ---
Name       : Asha
Subject    : Maths
Marks      : 87.5 / 100.0
Percentage : 87.5%
```

Notice how `strip()` cleaned the extra spaces around the name, and how `:.1f` and `:.1%` shaped the numbers.

8 Common errors and good practices

Error / habit	What it means
`NameError`	Using a variable name that has not been defined.
`ValueError`	Trying int("hello") or float("abc").
`TypeError`	Combining incompatible types, e.g. "Age: " + 19 (convert with str(19) or use an f-string).
`Case sensitivity`	input() is correct; Input() is a different (undefined) name.

- Use `.strip()` for text fields where leading/trailing spaces are not wanted.
- Use f-strings for readable output.
- Use `float()` for decimal input and `int()` for whole-number input.

Spot the bug: what goes wrong, and how do you fix it?

Buggy line	Problem	Fix
`age = input("Age: ") then age + 1`	TypeError: age is text, 1 is a number	`age = int(input("Age: "))`
`int("18.5")`	ValueError: not a whole-number literal	`float("18.5")`
`Input("Name: ")`	NameError: Python is case-sensitive	`input("Name: ")`

9 Practice tasks and recap

Quick-check answers

#	Question	Answer
1	" Python class ".strip() returns?	`"Python class"`
2	type() of input() when the user types 25?	`<class 'str'>`
3	What does int("25.5") do?	`raises ValueError`
4	print("2026", "10", "13", sep="-") shows?	`2026-10-13`
5	f"{marks:.2f}" with marks = 87.456?	`87.46`
6	f"{percentage:.1%}" with percentage = 0.875?	`87.5%`
7	Which line raises a TypeError?	`"Age: " + 19`

Model solutions

Your code may differ and still be right. Compare the **idea**, not the wording.

Task 1: name and city greeting

```
name = input("Name: ").strip()
city = input("City: ").strip()
print(f"Hello {name}, welcome to {city}!")
```

Task 2: sum of two whole numbers

```
a = int(input("First number: "))
b = int(input("Second number: "))
print("Sum:", a + b)
```

Task 3: total with 2 decimal places

```
price = float(input("Price: "))
quantity = int(input("Quantity: "))
print(f"Total: {price * quantity:.2f}")
```

Task 4: decimal shown as a percentage

```
x = float(input("Decimal between 0 and 1: "))
print(f"{x:.1%}")
```

Task 5: clean a name with strip()

```
name = input("Name: ").strip()
print(name)
```

Task 6: full name in title case

```
first = input("First name: ").strip()
last = input("Last name: ").strip()
print(f"{first} {last}".title())
```

Task 7: length, upper, lower

```
text = input("Sentence: ")
print(len(text), text.upper(), text.lower())
```

Task 8: marks percentage

```
marks = float(input("Marks obtained: "))
total = float(input("Total marks: "))
print(f"{marks / total:.1%}")
```

Task 9: split three foods

```
foods = input("Three favourite foods: ").split(",")
print(foods)
```

Task 10: add course to the report

```
course = input("Course: ").strip()
# add this line to the report:
print(f"Course     : {course}")
```

Recap in one line each

Question	Answer
What type does input() return?	A **str** (text), always.
Why use int() or float()?	input() gives text; arithmetic needs numbers, so we convert.
What does .strip() remove / keep?	Removes whitespace at the **start and end**; keeps the text and any inner spaces.
What does :.2f mean?	Show the number as fixed-point with **two decimal places**.
Formatting vs changing a value?	Formatting only changes the **display**; the variable keeps its original value (e.g. 87.456).

Learning checklist: tick each box when you can do it confidently

- [ ] Read user input with input() and clean it with strip().
- [ ] Convert input with int() and float(), and handle invalid numbers with try / except.
- [ ] Control output with print() using sep= and end=.
- [ ] Build readable messages with f-strings and formats such as :.2f and :.1%.
- [ ] Use basic string operations: +, *, len(), and indexing.

NEXT LESSON **Day 3**: Python Data Structures: lists, tuples, sets and dictionaries.